Level 2
Hackathon – Fundamentals of Web Development
College Event Registration Portal

Student Details

Student Name	
Register Number	
Department	
ID	
College	
Date of Submission	

Git-Hub Link:

1. Problem Statement

Colleges host many events every term – technical contests, workshops, cultural programs and sports meets – but the details are usually spread across notice boards, WhatsApp groups and posters. Students miss events because they never saw the announcement, registrations are collected on paper or in scattered forms, organizers cannot easily tell how many people have signed up, and the same student can end up registering twice for one event. When an event has limited seats there is no clear way to stop registrations once it is full.

The College Event Registration Portal solves this by giving students one interactive web page where they can explore upcoming events, read full details, register through a validated form, see the events they have registered for, cancel when permitted, and see live registration counts and remaining seats for every event.

2. Objectives

- Display upcoming college events dynamically with title, date, venue, description and category.
- Let students search events by name and filter them by category and date.
- Show detailed information for each event on demand.
- Provide a registration form with mandatory field validation and email validation.
- Prevent duplicate registrations for the same event and respect each event's capacity limit.
- Show a student’s registered events and allow cancellation where permitted.
- Store event and registration details in the browser using Local Storage so data persists across visits.
- Display registration counts and remaining seats for each event, updating instantly without a page reload.

Expected outcome: a fast, single-page portal that replaces scattered announcements and manual sign-up sheets with one reliable, easy-to-use registration system.

3. Architecture

3.1 Application Structure and Navigation Flow

The portal is a single-page application built from one HTML shell: a header with live counters, a sticky tab bar (Events, Register, My Events) and one <main> mount point. On load, a single **loadData()** function reads the events and registrations from Local Storage, seeding sample events the first time the app runs. Each tab is a separate render function that turns the shared in-memory **state** object into HTML and injects it into <main>, so switching tabs, searching, filtering, registering and cancelling never reload the page.

Application flow diagram:

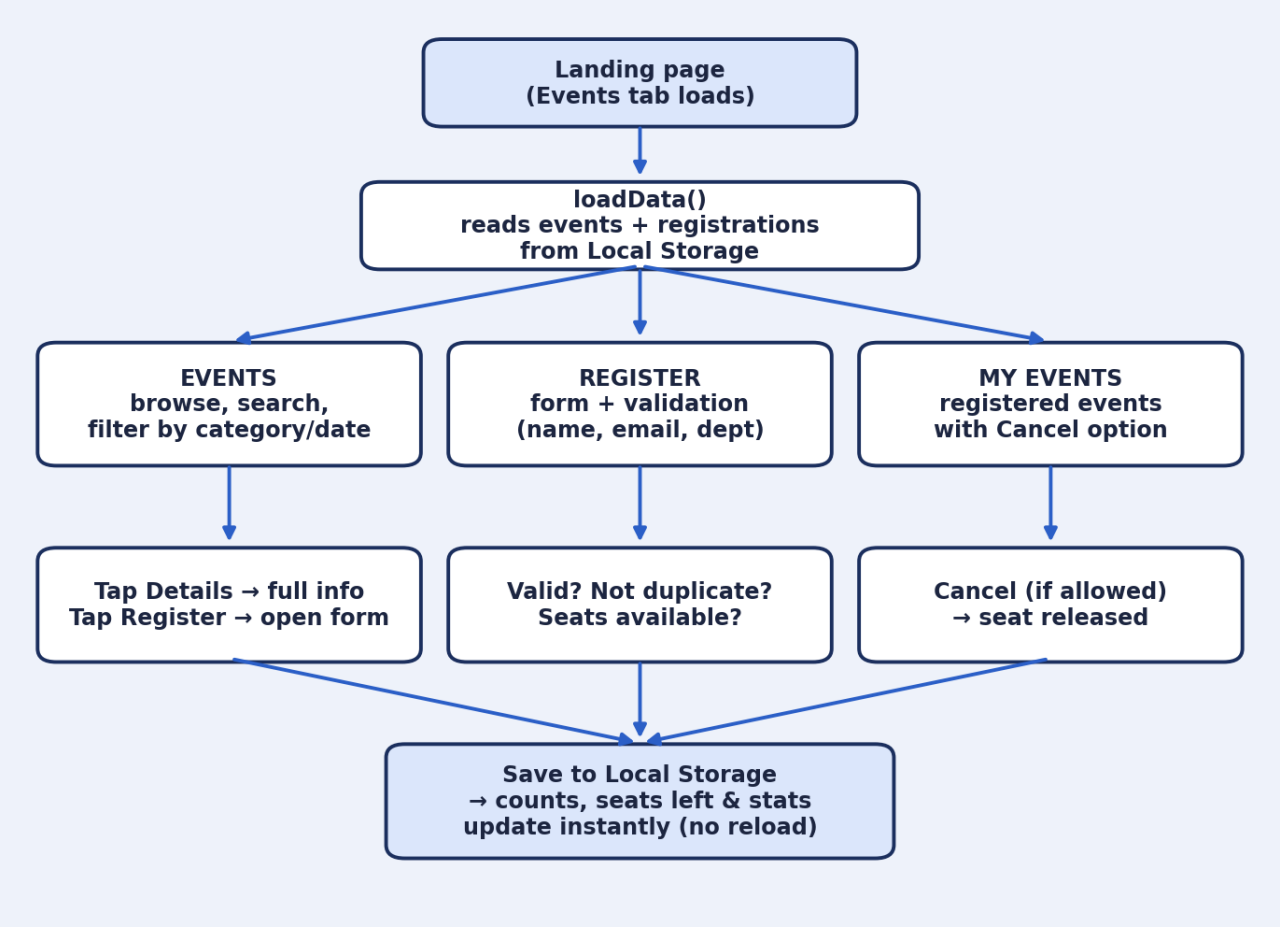


Fig. 1 — Application flow: data load, tab navigation and the interactions each tab supports.

3.2 Technologies Used

Technology	Role in the application
HTML5	One semantic document (header, nav, main) into which every view is rendered; form elements (input, select, date) for registration and filtering.
CSS3	Custom properties for the colour palette, Flexbox/Grid for layout, a sticky navigation bar, and a fluid single-column design that works on phones.
Vanilla JavaScript (ES6+)	Template-literal rendering, DOM event listeners, form validation with a regular expression, array methods for search/filter/sort, and Local Storage persistence.
Local Storage (Web Storage API)	Stores events, registrations and the student's email in the browser so data survives page refreshes.

4. Practical Implementation

The application was developed as one self-contained file (index.html) using only HTML, CSS and JavaScript, with no framework or server. A central **state** object holds the events, registrations, active tab and filter values. Every user action updates this state, saves any change to Local Storage, and calls **render()** to redraw the current view and the header statistics. Capacity is enforced by comparing the number of registrations for an event with its capacity, and duplicates are prevented by checking whether the same email is already registered for the same event.

4.1 Features Implemented

Feature	Purpose / Description
Dynamic Event Listing	Events are rendered from data with title, category tag, date, venue and time, sorted by date.
Search and Filter	Search by event name, filter by category dropdown and by date; results update instantly as the student types.
Event Details	“Details” expands a card to show the full description, date, time, venue and capacity.
Registration Form	Collects event, name, email and department; every field is mandatory and the email is checked against a valid pattern, with inline error messages.
Duplicate Prevention	A second registration with the same email for the same event is rejected with a clear message.
Capacity Limits	Each event has a seat limit; remaining seats are shown, low seats are highlighted and a full event shows “Event Full” with Register disabled.
My Registered Events	Lists all events registered by the student (matched by email) with an empty-state message when there are none.
Cancel Registration	Registered events can be cancelled up to one day before the event date; after that the button is disabled.
Local Storage Persistence	Events, registrations and the student’s email are saved in the browser and reloaded on the next visit.
Registration Counts	Every event card shows how many students have registered; the header shows total events, the student’s registrations and total seats left.
Responsive Design	Fluid single-column layout with a maximum width, sticky tab bar and wrapping filter controls for phones and desktops.

4.2 Source Code

The important sections of the source code are shown below; the complete file (index.html) is submitted along with this report.

HTML

HTML provides the page structure: the header with live statistics, the tab navigation and the single <main> element that every view renders into.

```
<header>
  <h1>College Event Registration Portal</h1>
  <div class="stats">
    <div><b id="sEvents">0</b><span>Events</span></div>
    <div><b id="sMine">0</b><span>My Registrations</span></div>
    <div><b id="sSeats">0</b><span>Seats Left</span></div>
  </div>
</header>
<nav id="tabs">
  <button data-tab="events" class="active">Events</button>
  <button data-tab="register">Register</button>
  <button data-tab="mine">My Events</button>
</nav>
<main id="main"></main>
```

CSS

CSS defines the palette once as custom properties, styles the event cards, seat indicators and buttons, and keeps the tab bar sticky while scrolling.

```
:root{--navy:#1b2f5e;--blue:#2b5fc7;--line:#dde4f0;--ok:#1f8a4c;--bad:#c0392b;--warn:#b7791f}
nav{display:flex;position:sticky;top:0;background:#fff;border-bottom:1px solid var(--line)}
nav button.active{color:var(--blue);border-bottom-color:var(--blue);font-weight:600}
.card{background:#fff;border:1px solid var(--line);border-radius:12px;
      padding:14px;margin-bottom:12px}
.seats{font-size:12.5px;font-weight:600;color:var(--ok)}
.seats.low{color:var(--warn)}  .seats.full{color:var(--bad)}
button.btn:disabled{opacity:.45;cursor:not-allowed}
main{max-width:760px;margin:0 auto;padding:16px}
```

JavaScript

JavaScript implements the logic: Local Storage helpers, search/filter, validation, duplicate and capacity checks, and cancellation.

```
// Local Storage helpers (try/catch so a storage failure never crashes the page)
function load(key, fallback){
  try{ const v = localStorage.getItem(key); return v ? JSON.parse(v) : fallback; }
  catch(e){ return fallback; }
}
function save(key, val){
  try{ localStorage.setItem(key, JSON.stringify(val)); }
  catch(e){ toast("Storage unavailable"); }
}

const count = id => state.regs.filter(r => r.eventId === id).length;   // registrations per event
const left  = e  => e.capacity - count(e.id);                          // seats remaining

// Search by name + filter by category and date
const list = state.events.filter(e =>
  (!q || e.title.toLowerCase().includes(q)) &&
  (state.category === "All" || e.category === state.category) &&
  (!state.date || e.date === state.date));

function validate(){
  // ... read #rEvent, #rName, #rEmail, #rDept ...
  if(!ev)    errs.eEvent = "Please select an event.";
  if(!name)  errs.eName  = "Name is required.";
  if(!email) errs.eEmail = "Email is required.";
  else if(!/^[^\s@]+@[^\s@]+\.[^\s@]{2,}$/.test(email))
    errs.eEmail = "Enter a valid email address.";
  if(!dept)  errs.eDept  = "Department is required.";
}
function submitRegistration(){
  const data = validate(); if(!data) return;
  const ev = state.events.find(e => e.id === data.eventId);
  if(isRegistered(data.eventId, data.email)){       // duplicate check
    $("#eEmail").textContent = "Already registered for this event."; return; }
  if(left(ev) <= 0){                                // capacity check
    $("#eEvent").textContent = "Sorry, this event is full."; return; }
  state.regs.push(data); save("cerp-regs", state.regs);
  state.myEmail = data.email; save("cerp-my-email", data.email);
  state.tab = "mine"; toast("Registration successful"); render();
}
function cancelRegistration(id){
  const me = state.myEmail.toLowerCase();
  state.regs = state.regs.filter(r => !(r.eventId === id && r.email.toLowerCase() === me));
  save("cerp-regs", state.regs);
  toast("Registration cancelled"); render();
}
```

4.3 Project Execution

To run the application, save the project file as **index.html** and open it in any modern web browser (or open the GitHub Pages link). No installation, server or internet connection is required.

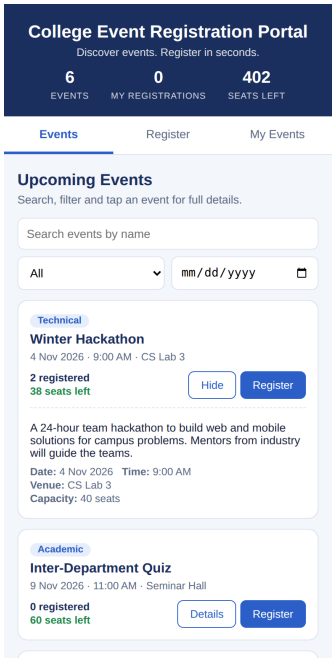
Working process, from input to output:

- The Events tab opens by default. **loadData()** reads Local Storage; on the first visit it seeds the sample events so the page is never empty.
- The student types in the search box or picks a category/date; the list is re-filtered instantly.
- Tapping **Details** expands the event card with its full description, venue, time and capacity.
- Tapping **Register** opens the form with that event selected. The student enters name, email and department.
- On **Submit**, the form is validated (mandatory fields, email format), then checked for duplicate registration and available seats.
- If valid, the registration is saved to Local Storage, counts and seats-left update, and the student is taken to **My Events**.
- In My Events the student can **Cancel** a registration where permitted; the seat is released and the counters update.

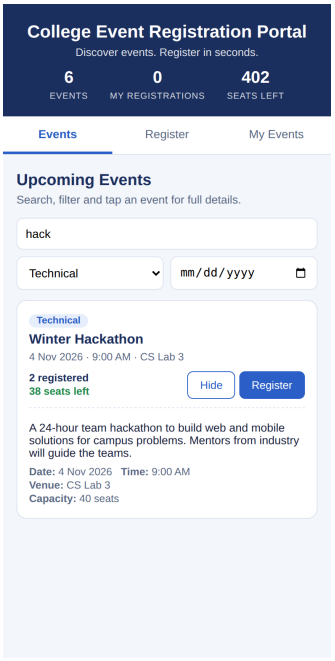
5. Output Screenshots

Screenshots below are taken directly from the working application.

Screenshot 1 – Event Listing with Details | Screenshot 2 – Search and Filter

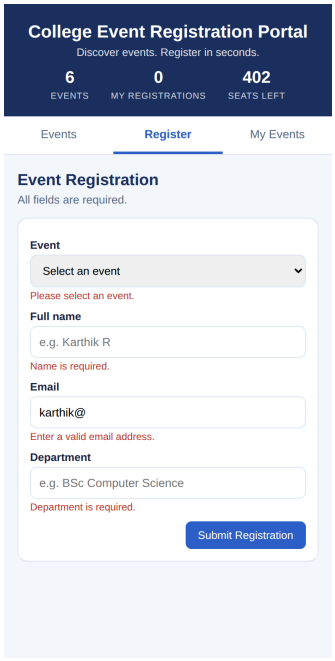


Events tab: dynamic event cards with registration counts, seats left and an expanded detail view.

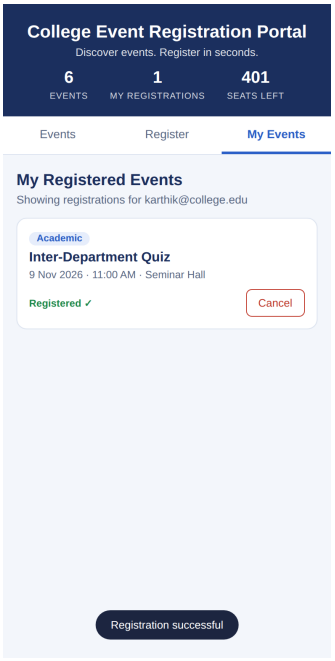


Searching "hack" and filtering by category Technical shows only the matching event.

Screenshot 3 – Form Validation | Screenshot 4 – My Registered Events

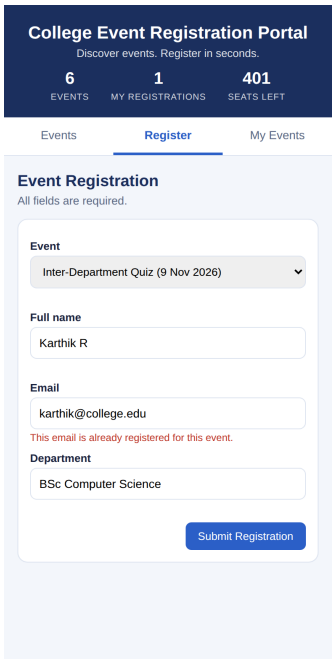


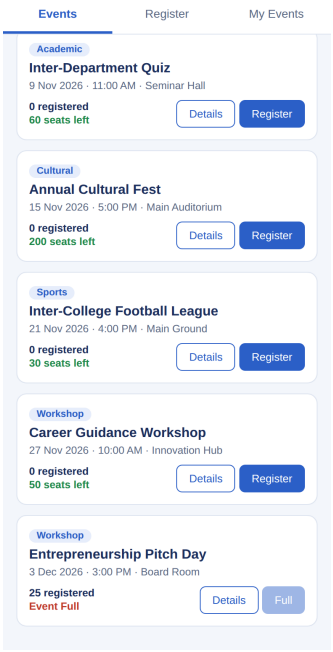
Registration form showing mandatory-field and email validation errors.



My Events tab listing the registered event with a Cancel button; header counters updated.

Screenshot 5 – Duplicate Prevention | Screenshot 6 – Capacity Limit

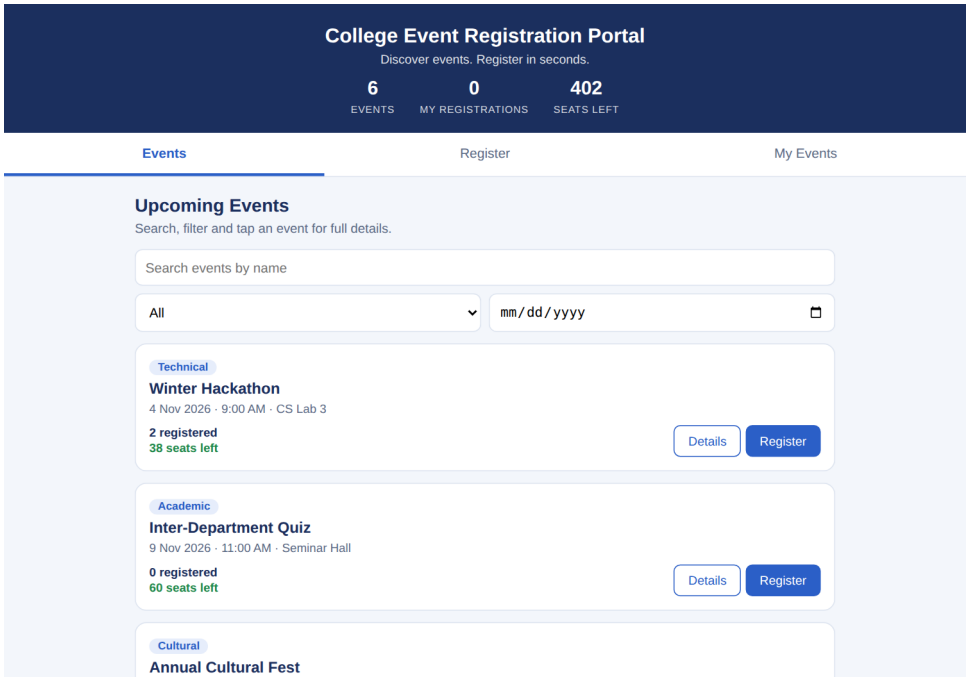




Registering again with the same email for the same event is blocked with a clear message.

A fully booked event shows "Event Full" and its Register button is disabled.

Screenshot 7 – Responsive Desktop View



The same layout adapts from a narrow phone screen (above) to a desktop browser window.

6. Technical Explanation

HTML

The document is one static shell – a header with live statistics, a sticky navigation bar and a single <main> element. The markup for each tab is generated as a JavaScript template string and injected into #main, instead of writing separate HTML pages. Form controls (input, select, date) collect search, filter and registration data.

CSS

CSS custom properties in :root define the colour palette once so every component reuses it. Flexbox and Grid arrange the header counters, filters and card rows inside a centered column with a 760px maximum width, and position:sticky keeps the tab bar visible while scrolling. Disabled and “full” states are styled so unavailable actions are obvious.

JavaScript

A single **state** object stores the events, registrations, active tab, search text, category, date and the student’s email. Every action mutates state, persists changes with Local Storage, and calls **render()**, which regenerates the HTML for the current tab and re-attaches event listeners. Concepts used include DOM manipulation, event handling, array methods (filter, find, some, sort), template literals, regular expressions for email validation, JSON serialization, and try/catch error handling for storage.

Key Functions

- **loadData()** – loads or seeds events and registrations from Local Storage.
- **render() / renderEvents() / renderRegister() / renderMine()** – turn the current state into HTML for the active tab.
- **validate()** – checks mandatory fields and the email format and shows inline errors.
- **submitRegistration()** – applies duplicate and capacity checks, then saves the registration.
- **cancelRegistration()** – removes a registration and releases the seat.
- **attachHandlers()** – re-wires all listeners after each render, since rendering replaces the DOM nodes.

7. Conclusion

The College Event Registration Portal replaces scattered notices and manual sign-up sheets with one page where students can discover events, read details, register with validated input, track and cancel their registrations, and see live seat availability. The project reinforced how a single state object with small render functions can power a complete interactive application without a framework, how Local Storage can persist data in the browser, and how validation, duplicate prevention and capacity checks keep the data reliable.

8. Future Enhancements

- Backend and database (Node.js or Firebase) so registrations are shared across devices instead of one browser.
- Secure student login and OTP-based email verification to stop fake registrations.
- Admin dashboard for creating and editing events and exporting registration lists to CSV or Excel.
- Waiting list that automatically promotes students when someone cancels.

- Email, SMS or push reminders before events, and QR-code tickets for check-in.
- Calendar export (.ics) and event feedback or ratings after each event.